JEWEL PORTAL · ENGINEERING BRIEFING

Drawing the Line

The portal now does the job we set out to build. This briefing shows, with measurements from the codebase itself, why the right next investment is consolidation — not another feature — and what the consolidation plan looks like.

22 August 2026 snapshot 39d00b7 automated audit, tools/refactor

The position in four numbers

187,726
lines of application code across 2,573 files

62%
of that code sits in the 385 files that exceed our 100-line file standard

181 / 411
API operations rely on sign-in alone, with no per-operation permission rule

7,601
duplicated lines (4.2%) across 637 copy-paste clone pairs

Measured on the current main branch with the audit tooling now committed to the repository, so every figure here can be re-produced — and tracked — with one command.

What we built, and why it was the right trade

In a short window the portal has grown to cover the business end to end: projects, valuations and claims, variations, work orders, bid packages and tenders, labour and cost tracking, cash forecasting, Xero reconciliation, mailbox intake with triage, document control, and an AI assistant that operates across all of it. Eighty-one screens, four hundred API operations.

That pace was bought the usual way: by building each feature into the nearest page rather than pausing to generalise. It was the right trade while we were finding out what the product needed to be. The measurements below show where the bill now sits — and why it is cheapest to pay it now.

Where the risk is concentrated

Our standard says no file over 100 lines, because small files are what keep change cheap and safe. 2,188 of our 2,573 files meet it. The problem is that they hold barely a third of the code. The other two thirds has pooled in a small number of very large files — and those files are exactly where every new feature, fix, and AI-assisted change has to land.

2,188 files within the standard — 71,711 lines

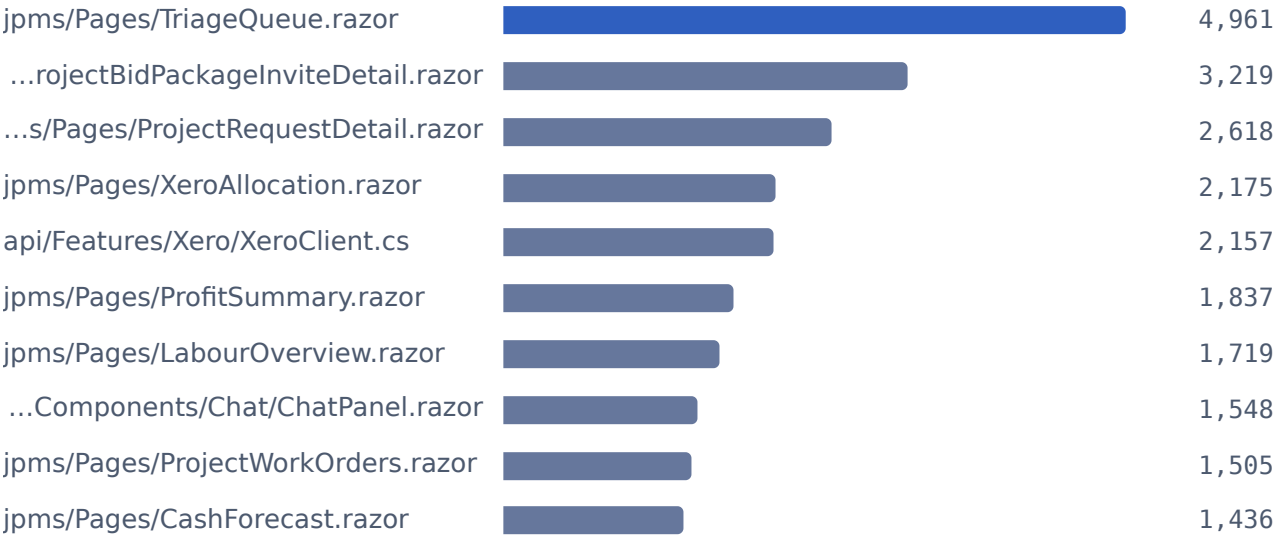
375 files over 100 lines — 92,840

the 10 largest files — 23,175

Ten files hold an eighth of the entire codebase. The single largest, the email triage screen, is 4,961 lines — nearly fifty times the standard.

The ten largest files

Lines per file · the standard is 100



Screens average 544 lines each across 79 pages; eleven screens exceed 1,000 lines. 602 individual functions exceed our 30-line function standard.

Why this matters commercially rather than aesthetically: a change inside a 5,000-line file cannot be reviewed with confidence, cannot be tested in isolation, and collides with any other change being made to the same screen — we have already had two pieces of work land on the triage screen at the same time and nearly overwrite each other. Each feature added this way is a little slower and a little riskier than the last. That curve only bends one way until we intervene.

Duplication compounds the same risk. 637 cloned blocks — 603 of them copied *across* files — mean one bug fixed in one place quietly survives in its copies. The heaviest clone families are API endpoint boilerplate repeated over ~400 endpoints and document renderers sharing copied scaffolding.

The security picture

The API has a genuinely good skeleton: every operation follows the same pattern of resolving the signed-in user, then checking permission, then validating input, before touching data. The gap is coverage, not design.

181 of 411 operations — 44%, mostly data reads — check that a user is signed in but apply no rule about *which* user may perform them. Today any authenticated member of staff can read project financials, directory listings, to-dos, and mailbox search results regardless of role. Fixing this is mechanical precisely because the gate pattern already exists — but it must be done before the exposure widens further.

The lighter findings: administrator database connection strings are assembled inside the Azure setup scripts (they belong in Key Vault references), automated test coverage is thin at 31 test files for 188k lines, and a full stack-specific sweep — auth boundary, third-party token handling for Xero and Microsoft Graph, upload paths, dependency patch level — is scheduled as a stage of the plan below rather than left as good intentions.

Why the line is here

Three things are true at once, and together they pick this moment.

The feature set has converged. The portal now covers the workflows the business runs on. What remains is tweaking, not invention — meaning a consolidation pause costs us features we largely weren't going to build anyway.

The cost of waiting rises with every feature. Each new feature built on today's structure adds to the very files that must later be taken apart, making the refactor strictly more expensive next quarter than this one — while itself being slowed by the structure it lands in.

The refactor is unusually well-conditioned right now. The audit shows real assets to consolidate around: a consistent API pattern, a shared-component layer the screens already use (our loading gate appears in 73 files, our modal in 56). This is a codebase that half-knows its own architecture. We are extracting a design that is already visible, not inventing one — which is what makes this a staged, verifiable programme rather than a risky rewrite.

The programme

The plan is committed to the repository as a staged playbook with tooling that makes it measurable. Two properties matter for oversight:

> **It is behaviour-preserving, and it cannot regress.** No rewrite, no migration, no downtime: screens are decomposed in place with identical behaviour. An automated audit now runs on every change and *fails* any change that makes the numbers worse — so from the day the baseline was committed, every figure in this briefing can only improve. Progress is visible in the same numbers, week by week.

STAGE	WHAT HAPPENS	WHAT LEADERSHIP SEES
`0 · Baseline`	Audit committed; ratchet gate turned on in CI	This briefing's figures, frozen as the floor
`1–2 · Model`	Map every screen to the user stories it serves; classify screens and widgets into a small reusable taxonomy	A one-page map of the product
`3 · Contract`	One lifecycle for every widget: load, empty, error, ready — one loading and transition language everywhere	Visibly consistent screen behaviour
`4–5 · Extract`	Largest files decomposed into shared widgets; clone families merged into single named concepts	Files-over-limit and duplication falling
`6–7 · Secure`	Permission rules on all 181 uncovered operations; full stack-specific security sweep of app and Azure infrastructure	Uncovered operations at zero; findings log
`8 · Design`	Widgets reconciled with the original Figma designs; colours, spacing and motion moved to named tokens	The product looking like the designs, everywhere at once
`9 · Hold`	Gate stays on permanently; new code meets the standard from birth	Numbers that only move one way

Bug fixes and small tweaks continue throughout — the freeze applies to net-new feature scope only. Because the audit and playbook are generic, the same pipeline becomes reusable on any future repository we stand up.

What the business gets

Faster delivery once the pause ends, because features will assemble from shared widgets instead of being hand-built into giant pages. Fewer regressions, because small files are reviewable and testable and fixes stop leaving copies of the bug behind. A closed permission model over company financial data. A product that matches its designs on every screen, because a fix to a widget fixes it everywhere. And a codebase that stays this way, because the gate makes the standard self-enforcing.

Method: figures produced by the audit pipeline at `tools/refactor` (file, function, duplication, naming, inventory and gate checks) against commit `39d00b7` of 22 Aug 2026; C# and Razor sources, excluding generated code. Duplication measured with jscpd at 70-token minimum. Function-length figures are heuristic. Every number is reproducible via `python3 -m audit.run_audit`.